

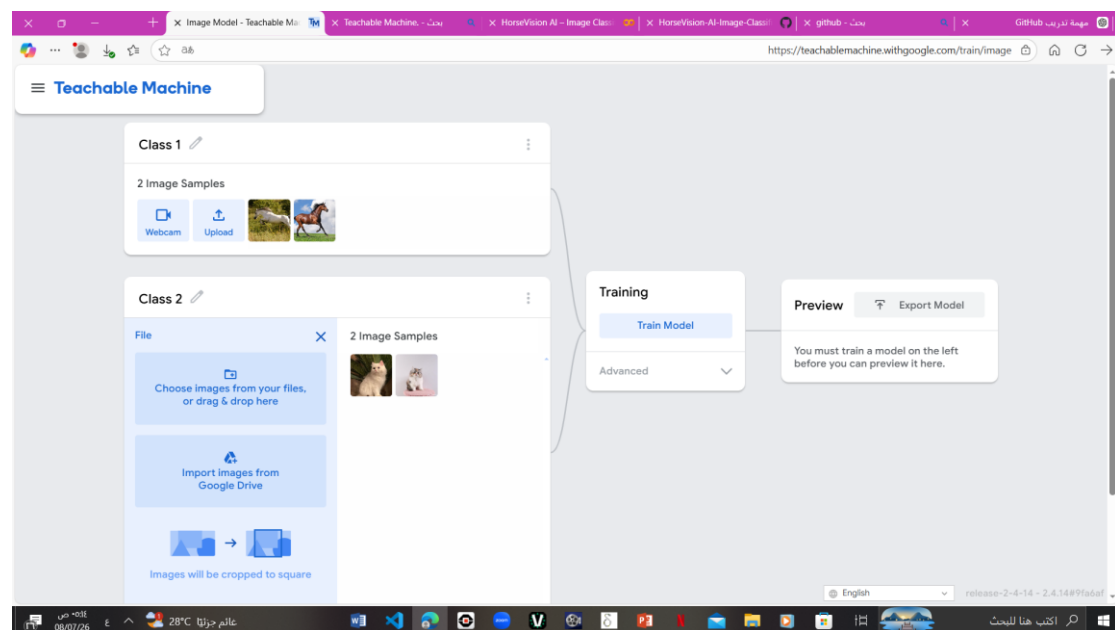
Step 1: Data Collection

First, images were collected to create the dataset.

The images were divided into different categories (classes). For example:

- Class 1: Horse images
- Class 2: Other images

The purpose of collecting different classes is to teach the AI model how to distinguish between them.



step 2: Training the Model Using Teachable Machine

Google Teachable Machine was used to train the image classification model.

The steps were:

1. Open **Google Teachable Machine**.
2. Select **Image Project**.
3. Create classes for the images.
4. Upload training images into each class.
5. Start the training process.

During training, the neural network learns the important features of each image, such as shapes, colors, and patterns.

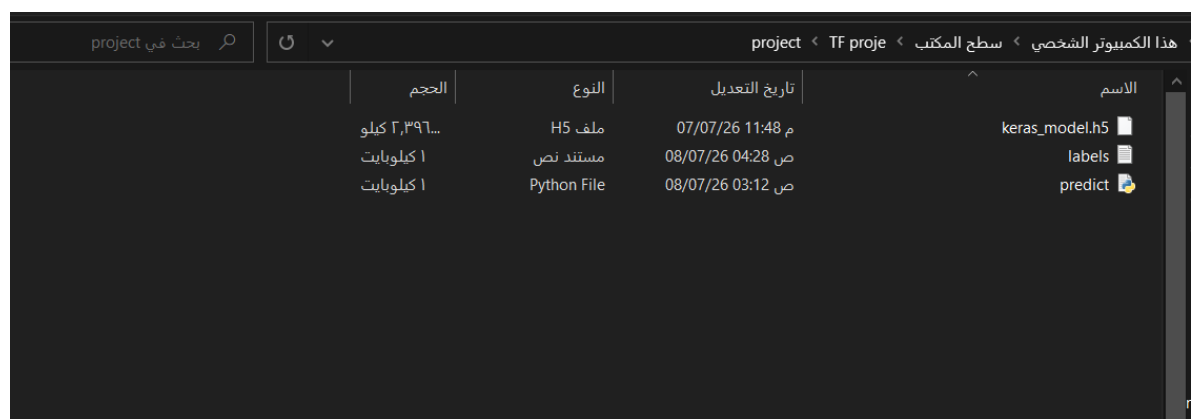
Step 3: Exporting the Trained Model

After completing the training:

1. Click **Export Model**.
2. Choose **TensorFlow/Keras model**.
3. Download the model files.

The exported files include:

- `keras_model.h5` → contains the trained neural network.
- `labels.txt` → contains the names of the classes.
-



step 1: Upload the Model Files

After training the model in **Teachable Machine**, download:

- `keras_model.h5`
- `labels.txt`

Upload them to Google Colab:

```
[ ] from google.colab import files

uploaded = files.upload()

من الملفات اختيار ملفات
keras_model.h5(n/a) - 2453432 bytes, last modified: 100% - ٢٠٢٦/٧/٧ done
labels.txt(text/plain) - 20 bytes, last modified: 100% - ٢٠٢٦/٧/٨ done
Saving keras_model.h5 to keras_model (1).h5
Saving labels.txt to labels (1).txt
```

Step 2: Install Required Libraries

Because the model was created using Teachable Machine, we use `tf_keras`:

```
!pip install tf_keras
Requirement already satisfied: urllib3<3,>=1.21.0 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow<2.21,>=2.20->tf_keras) (3.18)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow<2.21,>=2.20->tf_keras) (3.18)
Requirement already satisfied: certifi<2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow<2.21,>=2.20->tf_keras) (2026.6.17)
Requirement already satisfied: markdown<=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard==2.20.0->tensorflow<2.21,>=2.20->tf_keras) (3.10.2)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorboard==2.20.0->tensorflow<2.21,>=2.20->tf_keras) (11.3.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard==2.20.0->tensorflow<2.21,>=2.20->tf_keras) (0.7.0)
Requirement already satisfied: werkzeug<=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard==2.20.0->tensorflow<2.21,>=2.20->tf_keras) (3.1.8)
Requirement already satisfied: markupsafe<=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug<=1.0.1->tensorboard==2.20.0->tensorflow<2.21,>=2.20->tf_keras) (3.1.8)
Requirement already satisfied: markdown-it-py<=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow<2.21,>=2.20->tf_keras) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow<2.21,>=2.20->tf_keras) (2.20.0)
Requirement already satisfied: mdurl<=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py<=2.2.0->rich->keras>=3.10.0->tensorflow<2.21,>=2.20->tf_keras) (0.1.2)
Downloading tensorflow-2.20.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (620.7 MB)
620.7/620.7 MB 2.3 MB/s eta 0:00:00
Downloading keras-3.15.0-py3-none-any.whl (1.7 MB)
1.7/1.7 MB 39.2 MB/s eta 0:00:00
Using cached ml_dtypes-0.5.4-cp312-cp312-manylinux_2_27_x86_64.manylinux2014_x86_64.whl (327 kB)
327.1/327.1 kB 22.3 MB/s eta 0:00:00
Downloading tensorboard-2.20.0-py3-none-any.whl (5.5 MB)
5.5/5.5 MB 48.4 MB/s eta 0:00:00
Installing collected packages: protobuf, ml_dtypes, tensorboard, keras, tensorflow
Attempting uninstall: protobuf
Found existing installation: protobuf 4.25.9
Uninstalling protobuf-4.25.9:
Successfully uninstalled protobuf-4.25.9
Attempting uninstall: ml_dtypes
Found existing installation: ml-dtypes 0.3.2
Uninstalling ml-dtypes-0.3.2:
Successfully uninstalled ml-dtypes-0.3.2
```

Step 3: Import Libraries:

import `tf_keras` as `keras`

import `numpy` as `np`

from `PIL` import `Image`

Step 4: Load the Trained AI Model

The file `keras_model.h5` contains the trained neural network

```
[1]
10s
import tf_keras as keras

model = keras.models.load_model(
    "keras_model.h5",
    compile=False
)

print("تم تحميل النموذج بنجاح")
```

تم تحميل النموذج بنجاح

Step 5: Load Class Names

The file `labels.txt` contains the names of the classes.

```
[18]
✓ 0s
with open("labels.txt") as f:
    labels = [line.strip() for line in f]

print(labels)

... ['0 1', 'حصل Class 2']
```

Step 6: Upload an Image for Testing

Upload the image you want to classify:

```
[2]
✓ 21s
from google.colab import files

uploaded = files.upload()

اختار ملفات OSK.jpg
OSK.jpg(image/jpeg) - 34599 bytes, last modified: 100% - ٢٠٢٦/٧/٨ done
Saving OSK.jpg to OSK.jpg
```

Step 7: Prepare the Image

The mode

```
نسبة الثقة: 0.9999863

[20]
✓ 0s
image = Image.open("OSK.jpg").convert("RGB")

image = image.resize((224,224))

image_array = np.asarray(image).astype(np.float32)

# Normalize image values
image_array = (image_array / 127.5) - 1

# Add batch dimension
image_array = np.expand_dims(image_array, axis=0)

print(image_array.shape)

(1, 224, 224, 3)
```

`I` expects an image size of **224×224 pixels**.

Step 8: Predict the Image Class

The model analyzes the image:

```
[21]
✓ 0s prediction = model.predict(image_array)

print(prediction)

v 1/1 [=====] - 0s 70ms/step
[[9.999863e-01 1.369919e-05]]
```

This means:

- Class 0 probability = 99.9986%
- Class 1 probability = 0.0013%

Step 9: Get the Highest Probability Class:

```
[22]
✓ 0s class_index = np.argmax(prediction)

result = labels[class_index]

confidence = prediction[0][class_index]

print("Prediction:", result)
print("Confidence:", confidence)

v ... Prediction: 0 حسان
Confidence: 0.9999863
```

Step 10: Convert Confidence to Percentage

```
[23]
✓ 0s confidence_percentage = confidence * 100

print(
    "The image is:",
    result,
    "with confidence:",
    round(confidence_percentage, 2),
    "%"
)

v The image is: 0 حسان with confidence: 100.0 %
```